



Tools for building AI Agents

Week 4

Last Week

Week 3: Intro to AI Agents

Cont.' RAG, AI Agents, Reasoning

How to retrieve the most relevant context?

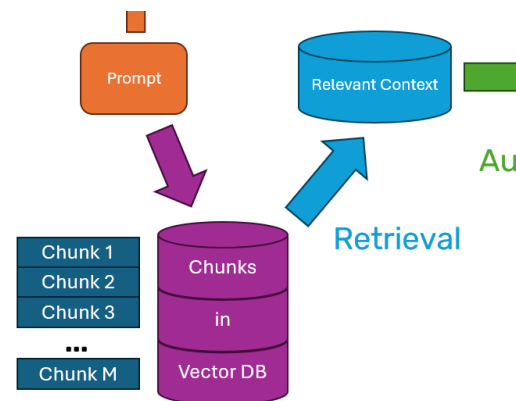
How to effectively embed documents?

Chunk Size

Overlap

How much context to retrieve?

Top K



What Content Should Be Included in a RAG Dataset?

Good for RAG

Long Forms: User Manuals, API Docs, etc.

Research papers and academic content

Code repositories & Documentation

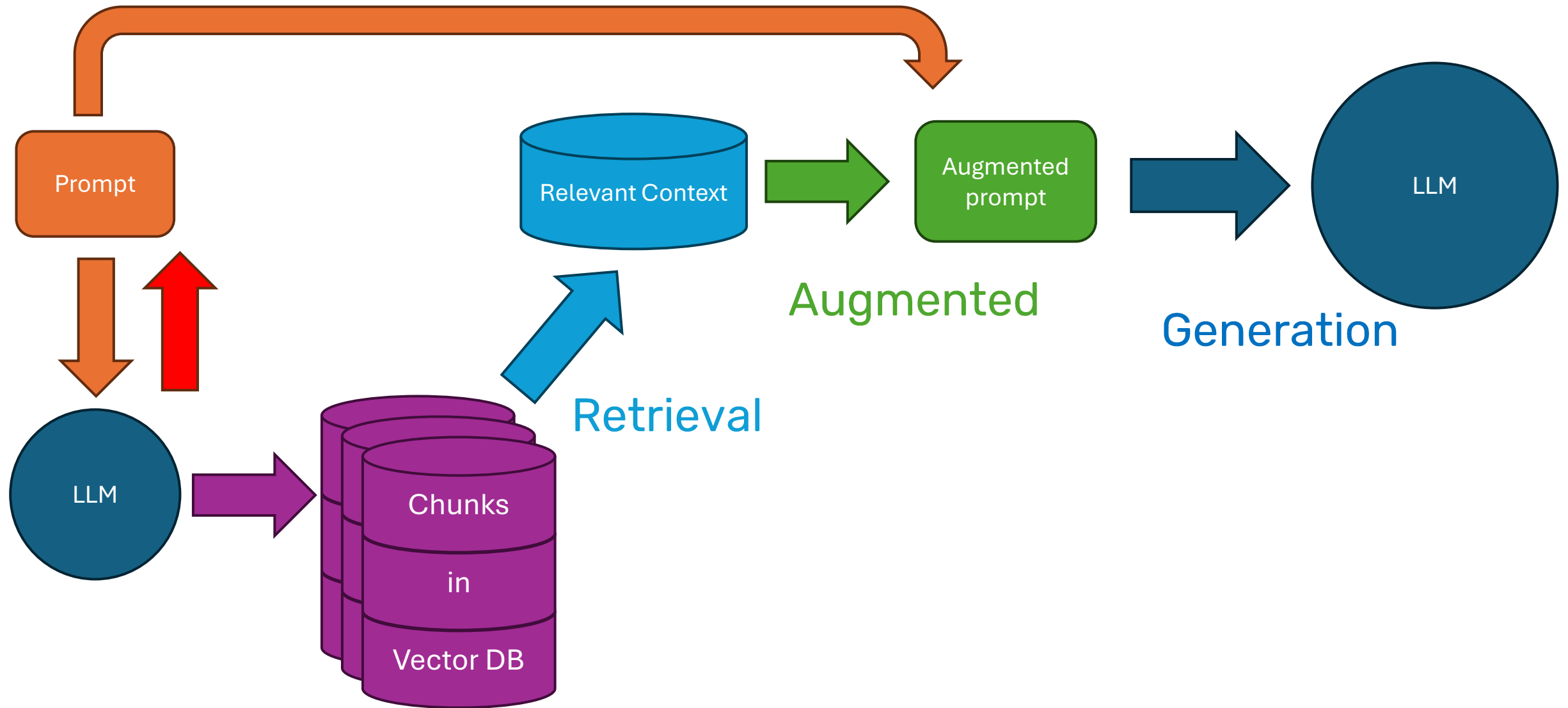
Bad for RAG

Real time data: Stock prices, traffic updates

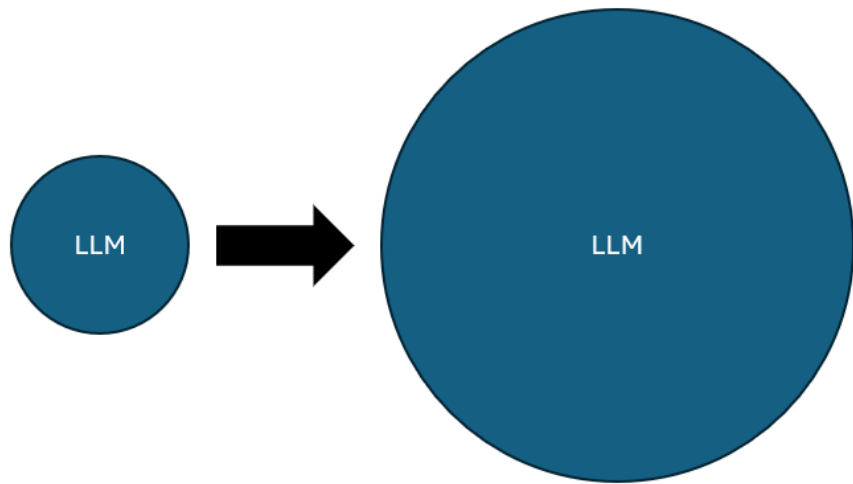
Inventory databases with quantities

Businesses locations for traffic app

A RAG Variant – Agentic RAG



Quality of LLMs



Reason - הגיון

This Week

Week 3: Intro to AI Agents

Cont.' RAG, AI Agents, Reasoning

Week 4: Tools for building AI Agents

LangChain, Azure Open AI Api, LLMod.ai



LangChain

[Products ▾](#)[Methods ▾](#)[Resources ▾](#)[Docs ▾](#)[Company ▾](#)[Pricing ▾](#)[Get a demo](#)[Sign up](#)

Applications that can reason. Powered by LangChain.

[Get a demo](#)[Sign up for free](#)

LangChain

What it is?

Framework designed to build applications using large language models (LLMs).

Popular use cases:
RAG, Autonomous agents.

Launched Oct 2022 on GitHub



<https://github.com/langchain-ai/langchain>

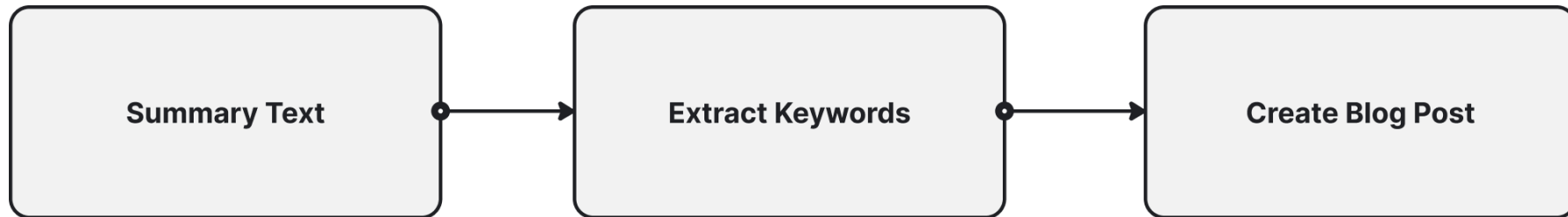
Jun 2023, Fastest growing open-source project



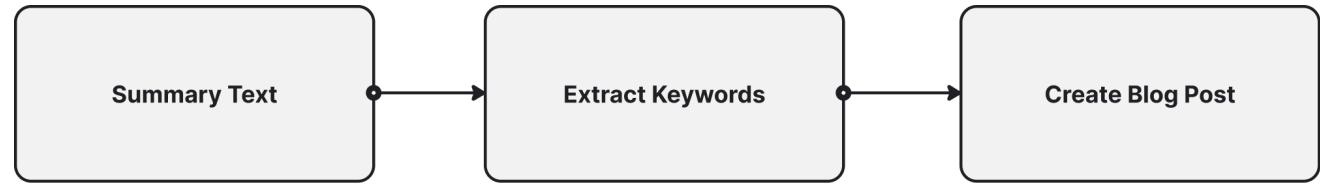
LangChain – Core Concepts

Chains

- 1 Chain multiple steps to create a more complex task



Chains



1 Chain multiple steps to create a more complex task

```
chain1 = LLMChain(llm=llm_gpt5mini, prompt=prompt_summary, output_key="summary")
chain2 = LLMChain(llm=llm_gpt5, prompt=prompt_keywords, output_key="keywords")
chain3 = LLMChain(llm=llm_llama3, prompt=prompt_blog, output_key="blog_post")

def combined_chain(input_text: str) -> str:
    # 1) summarize
    res1 = chain1.invoke({"input_text": input_text})
    summary = res1["summary"]

    # 2) extract keywords
    res2 = chain2.invoke({"summary": summary})
    keywords = res2["keywords"]

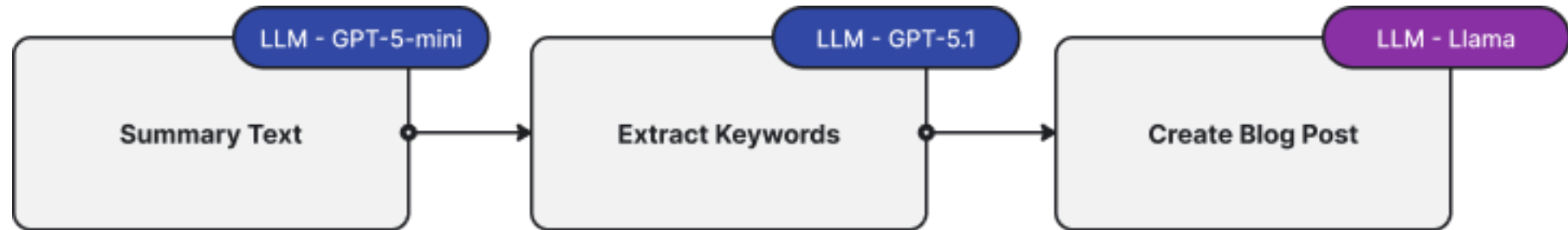
    # 3) create blog post
    res3 = chain3.invoke({"keywords": keywords})
    blog_post = res3["blog_post"]

    return blog_post
```

LLM Abstraction

- 2 Can use any LLM (OpenAI, Llama, etc)

Can use different LLMs in the same application



LLM Abstraction

- 2 Can use any LLM (OpenAI, Llama, etc)

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain_core.prompts import ChatPromptTemplate
import os

# --- LLMs ---
llm_gpt5mini = ChatOpenAI(model="gpt-5-mini")

llm_gpt5 = ChatOpenAI(model="gpt-5")

llm_llama3 = ChatOpenAI(model="llama-3")
```

LLM Abstraction

2

Can use any LLM (OpenAI, Llama, etc)

```
chain1 = LLMChain(llm=llm_gpt5mini, prompt=prompt_summary, output_key="summary")
chain2 = LLMChain(llm=llm_gpt5, prompt=prompt_keywords, output_key="keywords")
chain3 = LLMChain(llm=llm_llama3, prompt=prompt_blog, output_key="blog_post")

def combined_chain(input_text: str) -> str:
    # 1) summarize
    res1 = chain1.invoke({"input_text": input_text})
    summary = res1["summary"]

    # 2) extract keywords
    res2 = chain2.invoke({"summary": summary})
    keywords = res2["keywords"]

    # 3) create blog post
    res3 = chain3.invoke({"keywords": keywords})
    blog_post = res3["blog_post"]

    return blog_post
```

Prompt Templates

3 Prompt Templates: A better way to structure prompts

```
# --- Prompts ---
prompt_summary = ChatPromptTemplate.from_template(
    | "Summarize the following text in a few sentences:\n\n{input_text}"
    | )

prompt_keywords = ChatPromptTemplate.from_template(
    | "Extract the 5-10 most important keywords from this summary:\n\n{summary}"
    | )

prompt_blog = ChatPromptTemplate.from_template(
    | "Write a detailed blog post using these keywords:\n\n{keywords}"
    | )
```


System Prompts

4

Built in support for system prompts

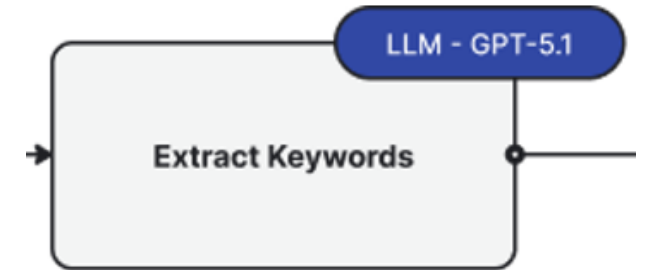
```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(model="gpt-5.1")

keyword_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are an assistant that extracts 5-10 concise, important keywords "
        "from a given text. Respond ONLY with a comma-separated list of keywords."
    ),
    ("user", "Text:\n\n{text}"),
])

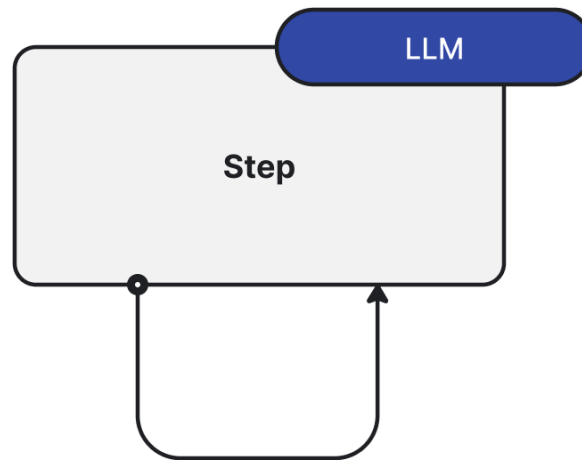
keyword_chain = LLMChain(llm=llm, prompt=keyword_prompt, output_key="keywords")

res = keyword_chain.invoke({"text": "Your long article or blog post goes here..."})
```



Memory

- 5 Memory: Full conversation or summary of conversation



Example

```
from langchain_core.messages import HumanMessage  
  
model.invoke([HumanMessage(content="Hi! I'm Bob")])
```

```
AIMessage(content='Hi Bob! How can I assist you today?')
```

```
model.invoke([HumanMessage(content="What's my name?")])
```

What will the AI answer?

```
AIMessage(content="I'm sorry, but I don't have access to personal information about users")
```

Example

```
from langchain_core.messages import AIMessage

model.invoke(
    [
        HumanMessage(content="Hi! I'm Bob"),
        AIMessage(content="Hello Bob! How can I assist you today?"),
        HumanMessage(content="What's my name?"),
    ]
)
```

What will the AI answer?

```
AIMessage(content='Your name is Bob! How can I help you today, Bob?')
```

Azure OpenAI API

Azure OpenAI API



What it is?

The Azure OpenAI API allows developers to programmatically access OpenAI's models for a variety of tasks like text generation, code completion, image creation, summarization, and more, all within the secure and scalable environment provided by Microsoft Azure.

It offers enterprise-grade security, compliance, and regional availability, making it suitable for businesses to build AI-powered applications.

Documentation:

<https://learn.microsoft.com/en-us/azure/ai-services/openai/>

Text Generation

What it is: Generate Text from Prompt

Course Model: gpt-5-mini

```
from langchain_openai import ChatOpenAI
from langchain.schema import SystemMessage, HumanMessage

# Create the model
llm = ChatOpenAI(model="gpt-5-mini")

# Call the model with messages
response = llm.invoke([
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="Write a haiku about recursion in programming.")
])

print(response.content)
```

Text Generation - Reasoning

```
from langchain_openai import ChatOpenAI
from langchain.schema import SystemMessage, HumanMessage

llm = ChatOpenAI(
    model="gpt-5-mini",
    reasoning={
        "effort": "medium", # 'minimal' | 'low' | 'medium' | 'high'
    },
    max_completion_tokens=1024, # optional; if omitted, API picks a default
)

response = llm.invoke([
    SystemMessage(content="You are a helpful assistant."),
    HumanMessage(content="Write a haiku about recursion in programming.")
])

print(response.content)
```


Text Generation

What it is: Generate Text from Prompt

Course Model: gpt-5-mini

Cost per 1M tokens:

Text tokens

Per 1M tokens • Batch API price ☐

Input

\$0.25

Cached input

\$0.025

Output

\$2.00

Embedding

What it is: Embed text into a numerical vector in a multi dimensional space

Course Models: text-embedding-3-small

```
from langchain_openai import OpenAIEmbeddings

# Create the embeddings client
emb = OpenAIEmbeddings(model="text-embedding-3-small")

# Single text -> vector
vector = emb.embed_query("Your text string goes here")

print(vector)          # this is the embedding list of floats
print(len(vector))     # embedding dimension
```

Embedding

What it is: Embed text into a numerical vector in a multi dimensional space

Course Models: text-embedding-3-small

Batching

```
from langchain_openai import OpenAIEmbeddings

# built-in batch size option
emb = OpenAIEmbeddings(
    model="text-embedding-3-small",
    batch_size=256,    # 👉 built-in batching
)

texts = [...]    # your list of strings

vectors = emb.embed_documents(texts)    # LangChain handles batching internally

print(len(vectors))
```

Embedding

What it is: Embed text into a numerical vector in a multi dimensional space

Course Models: text-embedding-3-small

Cost per 1M tokens:

Embeddings

Per 1M tokens • Batch API price ☒

Cost

\$0.02

LLMod.AI

Image Generation

What it is: System to use models, manage api keys & track budgets

The screenshot displays the LLMMod.ai Student Portal interface. The top navigation bar includes the LLMMod.ai logo and 'Student Portal' text. A breadcrumb trail shows 'Test Course 1 > Assignments > 1'. The main content area is titled 'Assignment 1' and 'Individual Assignment'. It features three summary cards: 'Assignment Period' (11/25/2025 to 12/2/2025), 'Budget' (\$5.00), and 'Cost Used' (\$0.0000 with a progress bar indicating 0.0% used). Below these is an 'API Key' section with a 'Current Key' field showing 'sk-...G9DQ' and a 'Rotate Key' button. The 'Base URL' is listed as 'https://api.llmod.ai'.

LLMod.ai
Student Portal

Test Course 1 > Assignments > 1

Assignment 1

Individual Assignment

Assignment Period
11/25/2025
to 12/2/2025

Budget
\$ 5.00

Cost Used
\$ 0.0000
0.0% used

API Key
Your personal API key for this assignment

Current Key: sk-...G9DQ [Rotate Key](#)

Base URL: https://api.llmod.ai



Test Course 1

Assignment 1

# Chat	0	0	\$0.0000
Total	0	0	\$0.0000

API Testing

Test your API integration and validate that everything works correctly. Use this to understand how to work with the API in your applications.

⚠ **Note:** API calls consume your assignment budget.

Test Configuration

API Key *

.....

Your API key is shown in the API Key section above

Model *

GPT-5-mini ▾

Prompt *

example prompt

▶ Run Test

LangChain Code

📋 Copy

JavaScript

Python



```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    api_key="sk-api",
    base_url="https://api.llmod.ai/v1",
    model="GPT-5-mini"
)

response = llm.invoke("example prompt")

print(response)
```

↑ Copy this code to use in your application



Student 5
student5@gmail.com

➔ Logout



LLMod.ai
Student Portal



Test Course 1 > Assignments > 1



Test Course 1

Assignment 1

# Chat	0	0	\$0.0000
Total	0	0	\$0.0000

API Testing

Test your API integration and validate that everything works.

⚠ Note: API calls consume your assignment budget.

Test Configuration

API Key *

.....

Your API key is shown in the API Key section above

Model *

GPT-5-mini

Prompt *

example prompt

▶ Run Test



Student 5
student5@gmail.com

Logout

LangChain Code

Copy

JavaScript

Python



```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    api_key="sk-api",
    base_url="https://api.llmod.ai/v1",
    model="GPT-5-mini"
)

response = llm.invoke("example prompt")

print(response)
```

Copy this code to use in your application

Individual Assignment

Due date: 7.6.2026 End Of Day

Weight: 10% of the course grade

Build a Medium Article RAG Assistant

A Retrieval-Augmented Generation assistant over a corpus of long-form Medium articles

Goal

Create a knowledgeable AI assistant specialised in **Medium articles** using a **Retrieval-Augmented Generation (RAG)** system. Your assistant must answer a set of questions **only from the provided article dataset**, without relying on any external information.

Unlike short texts, these articles are **longer** (typically hundreds to a few thousand tokens each). A single article will not fit neatly into one embedding, so the heart of this assignment is making good **chunking, overlap, and retrieval (top-k)** decisions.

Dataset

You will work with a dataset of English-language articles published on [Medium](#). The file provided to you is a single CSV containing roughly **7,600 English articles (~50 MB)**, drawn from the larger public Medium Articles dataset.

Provided file: [medium-english-50mb.csv](#)

Schema (CSV columns)

title, text, url, authors, timestamp, tags

1. Precise fact retrieval

Goal: The model must locate a single, specific article based on semantic criteria within the corpus.

Example: *"Find an article that reframes marketing as a conversation with readers, aimed at writers who find self-promotion uncomfortable. Provide the title and author."*

Explanation: The assistant should locate one concrete article and return the requested fields.

2. Multi-result topic listing (up to 3 results)

Goal: Return multiple distinct article titles that match a theme or topic.

Example: *"List exactly 3 articles about education. Return only the titles."*

Explanation: The assistant must retrieve multiple distinct articles, not multiple chunks of the same article. No need to support lists larger than 3.

3. Key idea summary extraction

Goal: Identify a relevant article and generate a concise summary of its main idea.

Example: *"Find an article that argues past pandemics (such as the bubonic plague) can spur innovation and recovery, and summarise its central argument."*

Explanation: The assistant should provide the key idea based on retrieved text chunks (not necessarily the whole article).

4. Recommendation with evidence-based justification

Goal: Recommend one relevant article and justify the choice.

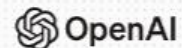
Example: *"I want practical, beginner-friendly advice on building habits that actually stick. Which article would you recommend, and why?"*

Explanation: The assistant should choose an article and provide a justification grounded in the retrieved text.

BUILD KNOWLEDGEABLE AI

Give agents **memory**

Fast retrieval. Accurate results. Lower costs.
Start in seconds.

[Start Building](#)[Get a Demo](#)

GET STARTED

[\[Claude Code\]](#)[\[Cursor\]](#)[\[Copilot\]](#)[\[Codex\]](#)[\[Gemini\]](#)[\[CLI\]](#)[\[MCP\]](#)[View Docs](#)

Questions?